Vision-5: Filtren på Pi:n

Första körningarna med höjd- och djupfiltret, och hur många bilder kameran levererar, 27 oktober 2026

Första testet efter PR #11 (filter, analyze_scans.py, bildrutor/s i sammanfattningen). Elias rum, dåligt ljus (ficklampa från mobilen, sedan stadigare ljus), Raspberry Pi 5 på powerbank (throttled 0x0), OAK-D Lite på USB HIGH, --facit 7340083497834, autofokus i alla körningar. Före testet hade Pi:n tappat wifi och syntes inte på nätverket. Löst genom att ansluta den igen. Rummet har annan bakgrund och annat ljus än köket, så siffrorna kan inte jämföras rakt av.

Sammanfattning

	Resultat	Status
Höjdfiltret på gamla loggar	Stoppar 60 av 62 fel, och 30 av 32 utan körningen 17/10 14:09	Uppmätt
Höjdfiltret vid pixelgränsen	Rätta läsningar var 1–28 px höga. Filtret kastade 33 av 35 rätta	Uppmätt, en uppställning
Felaktigt djup	Alla 15 inom 1,2 s från första läsningen, alla med låst lins. 0 i dag	Uppmätt
Bildrutor från kameran	1080p: 7,5 av 14,5/s. 4K: 2,7 av 4,8/s	Uppmätt. Orsaken (USB2) är beräknad
Autofokus, kod nära mitten	Landade på 84, 100 %	Uppmätt, nytt rum

Viktigaste lärdomarna: (1) **Höjden säger om en läsning är osäker, inte om den är fel.** Nära pixelgränsen är även rätta läsningar 1 px höga, så filtret stoppar allt där, rätt som fel. Det skyddar användaren men kortar räckvidden. (2) **Djupet är bara fel den första sekunden** efter att kameran startat. (3) **Bara ungefär hälften av kamerans bilder når Pi:n**, i både 1080p och 4K.

1. Alla körningar

Tid	Läge	Djup	px/modul	Autofokus	Bildrutor	Rätt kod	Fel	Kommentar
15:14	1080p	–	~1,5*	68–72	106	0 %	0	Mörkt, sedan ficklampa. Paketet för långt bort
15:17	1080p	0,19	2,64	84	11	100 %	0 av 27	Paketet närmare, kod nära mitten
15:18	4K	0,19	5,29	77–91	36	97 %	0 av 35	Samma uppställning
15:21	1080p	–	–	81	59	0 %	0	Flyttat bort, bortom gränsen
15:23	1080p	0,22	2,29	82	16	25 % / 0 %	0 av 9	Med --filter: alla 9 rätta avvisade
15:24	1080p	0,22	2,29	82	56	52 % / 4 %	0 av 35	Utan filter: 33 av 35 rätta hade avvisats

Rätt kod = andel bildrutor efter 10 s uppvärmning med facit, före / efter höjdfiltret. * Beräknat ur bilden (kodens bredd ~140 px), ingen läsning.

2. Fynd

1. Höjdfiltret på alla gamla loggar

Regel	Alla loggar (62 fel)	Utan 17/10 14:09 (32 fel)	Kastar rätta (av 1 409)
andel < 15 %	60 (97 %)	30 (94 %)	135 (10 %)
höjd < 30 px	60 (97 %)	30 (94 %)	133 (9 %)
andel < 15 % och höjd < 20 px	58 (94 %)	28 (88 %)	112 (8 %)

Filtret håller även utan körningen med flest fel. Andel och pixlar ger nästan samma resultat, eftersom koderna var ungefär lika stora i alla körningar. Regeln som skyddar korta koder kostar två fel till.

2. Höjden nära pixelgränsen

På 0,22 m (2,29 px/modul) var alla 44 läsningar rätt, men 42 av dem var tunna. Höjderna låg på 1–28 px (höjd/bredd 0,5–13 %), och flera var exakt 1 px, precis som felläsningarna i köket. En lägre gräns hjälper alltså inte: vid 1 px finns ingen skillnad i höjd mellan rätt och fel.

Slutsats (uppmätt, en uppställning): höjden mäter hur säker läsningen är, inte om den är rätt. Tunna läsningar uppstår nära pixelgränsen, både rätta och felaktiga. Filtret är en kvalitetsgräns som stoppar felläsningar genom att stoppa allt nära gränsen. Förslag: en tunn läsning ska ge ”lite närmare” i stället för tystnad.

3. Felaktigt djup kommer bara den första sekunden

analyze_scans.py hittade 15 orimliga djup (3–13 m) av 1 459 läsningar med djup, i sex körningar. Alla kom inom 1,2 s från första läsningen, och alla i körningar med låst lins, där koden läses direkt vid start. 9 kom från ”mitten” och 6 från ”kodens yta”, så reservrutan i depth_at är inte hela förklaringen. I dag, med autofokus, kom första läsningen efter flera sekunder och inget djup var fel. Hypotes: i det färdiga systemet är kameran igång hela tiden för hinderdetekteringen, och då försvinner problemet.

4. Kameran levererar bara hälften av bilderna

Läge	Kameran tar	Pi:n tar emot	Andel
1080p (15 fps)	14,5–14,8/s	7,5/s	~52 %
4K (5 fps)	4,8/s	2,7/s	~56 %

Beräknat behöver färg och djup tillsammans mer än USB2 klarar: ≈ 590 Mbit/s i 1080p och ≈ 570 Mbit/s i 4K, mot 480. Bilderna kan också försvinna på Pi:n medan den avkodar, så det är inte visat. Det avgörande testet är en blå USB3-port utan Y-adaptern, när strömförsörjningen finns. Skanningen påverkas inte nu (den avkodar högst 2–3 bilder/s), men hinderdetekteringen kommer att göra det.

5. Autofokus med koden nära mitten, igen

På 0,19 m landade autofokus på 84 och läste 100 %, i ett nytt rum med mönstrad tapet bakom. Det stämmer med 20/10: koden nära mitten ger rätt fokus.

3. Rättelser

- ”Filtret kastar bara ~10 % av de rätta, det märks knappt”: gäller på bekvämt avstånd. Vid pixelgränsen kastade det 94 %.
- ”En lägre gräns kan stoppa felen utan att slänga de rätta”: nej, rätta läsningar kan också vara 1 px höga.
- Granskarens misstanke om reservrutan i depth_at: stämmer bara delvis, 9 av 15 felaktiga djup kom från mitten.

4. Nästa steg

#	Steg	Varför
1	Beslut: ska en tunn läsning ge ”lite närmare” i stället för tystnad?	Säkerhet utan att tappa räckvidd, första biten av röstvägledningen
2	Testa en vara med kort streckkod	Den kända risken med höjdfiltret
3	Fokuslås vid träff (B1)	Del A är testad på Pi:n
4	USB3-test utan Y-adapter, när strömförsörjningen finns	Avgör om hälften av bilderna försvinner i USB2
5	Positionstestet, spaningsbesök i butiken	Planen